

SUGGESTIONS FOR RARITONE

Introduction

The Raritone Backend System is a robust foundation for a virtual fashion platform. Based on my experience during the internship, I have identified several enhancements that can elevate the platform to the next level. These suggestions focus on AI integration, performance optimization, security improvements, and deployment scalability.

1. Implement Full AI-Powered Virtual Try-On

Current State:

The backend has mock endpoints for AI try-on (POST /api/tryOnRoutes/tryon returning a mock session ID) and a TryOn model that stores original and generated images.

Suggestion:

Integrate a production-ready virtual try-on AI model such as VITON-HD, HR-VITON, or a commercial API (Zelig, [Vue.ai](#)). Replace the mock endpoints with real inference.

Benefits:

- Users can see exactly how a garment looks on their own body
- Significantly reduces return rates (estimated 30-50%)
- Increases purchase confidence and customer satisfaction

Implementation Path:

- Deploy AI model as a separate microservice
 - Use BullMQ queue to process try-on requests asynchronously
 - Emit real-time status updates via existing [Socket.IO](#) connection
-

2. Develop AI Avatar Generation

Current State:

The User schema has avatarImage and bodyScanImage fields. The Avatar routes exist, but the controller is not fully implemented.

Suggestion:

Build an AI avatar generator that converts a user's frontal photo into a customizable 3D avatar. Use generative models (Stable Diffusion, DALL-E, or a specialized fashion avatar model).

Benefits:

- Personalized avatars for virtual try-on and outfit visualization
- Increases user engagement and time spent on platform
- Opens monetization opportunities (premium avatar styles)

Implementation Path:

- Implement avatarController.js with createAvatar, getUserAvatars, deleteAvatar
 - Integrate with a generative AI API
 - Store generated avatars in Cloudinary and reference in Avatar collection
-

3. Build a Personalized Recommendation Engine

Current State:

The ProductMapping schema stores supportedBodyTypes, compatibility, and avatarCompatibility. User Measurement data (chest, waist, hip, height) is already collected.

Suggestion:

Create a recommendation engine that suggests products based on:

- User's body measurements (match with supportedBodyTypes)
- Purchase history and wishlist items
- Wardrobe items (e.g., "complete the look" suggestions)
- Browsing behavior

Benefits:

- Higher conversion rates through personalized suggestions
- Improved user retention
- Increased average order value

Implementation Path:

- Use collaborative filtering or a lightweight content-based algorithm
 - Cache recommendations in Redis for fast retrieval
 - Expose a new endpoint: GET /api/recommendations
-

4. Add Real-Time Features with [Socket.IO](#)

Current State:

[Socket.IO](#) is already set up in server.js with a basic tryon-request event.

Suggestion:

Expand real-time capabilities:

- **Order tracking:** Emit order-status events when order status changes (PLACED → SHIPPED → DELIVERED)
- **Avatar generation progress:** Notify client when AI avatar is ready
- **Try-on processing:** Emit tryon-status updates (processing → completed → failed)
- **Recommendation refresh:** Push new recommendations when user adds items to wardrobe

Benefits:

- Live feedback improves user experience
- Reduces need for client-side polling
- Makes the platform feel modern and responsive

Implementation Path:

- Define event names and payload structures
 - Emit events from controllers after state changes
 - Document events for frontend integration
-

5. Implement Redis Caching

Current State:

The product catalog is fetched from MongoDB on every request. No caching is in place.

Suggestion:

Integrate Redis to cache:

- Frequently accessed product lists (e.g., “new arrivals”, “top sellers”)
- User session data (optional, since JWT is stateless)
- API responses that change infrequently

Benefits:

- Reduces database load by 60-80%
- Response times drop from ~50ms to <5ms
- Enables horizontal scaling

Implementation Path:

- Set up Redis (local for dev, Redis Cloud or AWS ElastiCache for production)
- Use ioredis or redis npm package
- Cache product catalog with a TTL of 5 minutes; invalidate on product update/delete

6. Introduce Queue-Based Asynchronous Processing

Current State:

Heavy tasks like image upload/compression and AI processing are currently synchronous (except Sharp which is fast). Long-running AI tasks will block the event loop.

Suggestion:

Implement a queue system (BullMQ with Redis) for:

- AI avatar generation
- Virtual try-on inference
- Bulk email notifications (password reset, order confirmations)
- Report generation (admin analytics)

Benefits:

- API remains responsive even under heavy load
- Failed jobs can be retried automatically
- Workers can scale independently

Implementation Path:

- Install BullMQ and Redis
 - Create job producers in controllers
 - Create worker files that process jobs
 - Poll or use [Socket.IO](https://socket.io/) to notify completion
-

7. Enhance Security Features

Current State:

JWT, RBAC, rate limiting, Helmet, and Joi validation are implemented. Wishlist auth is fixed.

Suggestion:

- **Multi-Factor Authentication (MFA):** Add OTP via email or authenticator app for high-value accounts.
- **Email Verification:** Require users to verify email before accessing all features.
- **Refresh Token Rotation:** Implement refresh token versioning to detect reuse (theft).
- **API Key for Admin Services:** Secure internal admin endpoints with a second API key.

Benefits:

- Stronger protection against account takeover
- Compliance with security best practices (GDPR, etc.)
- Builds user trust

Implementation Path:

- Extend User schema with isEmailVerified and mfaEnabled
 - Use speakeasy or otplib for TOTP
 - Store refresh token versions in database
-

8. Implement Advanced Monitoring and Logging

Current State:

Morgan logs HTTP requests to the console. No centralized error tracking or performance monitoring.

Suggestion:

Integrate:

- **Sentry** for real-time error tracking
- **Prometheus + Grafana** for metrics (request rate, latency, error rate)
- **ELK Stack** (Elasticsearch, Logstash, Kibana) for log aggregation

Benefits:

- Faster incident detection and resolution
- Visibility into system bottlenecks
- Data-driven capacity planning

Implementation Path:

- Add Sentry middleware to app.js
- Expose a /metrics endpoint for Prometheus scraper
- Use winston for structured logging

9. Containerized Deployment with Docker and Kubernetes

Current State:

The project runs locally with node server.js. No containerization.

Suggestion:

Create a Dockerfile and docker-compose.yml for local development. For production, deploy on Kubernetes (EKS, AKS, or GKE).

Benefits:

- Consistent environment across dev/staging/production
- Easy horizontal scaling (kubectl scale)
- Zero-downtime rolling updates

Implementation Path:

- Write Dockerfile with multi-stage builds (small image size)
- Use Kubernetes manifests for deployment, service, and ingress

- Set up CI/CD with GitHub Actions to build and push images to a registry (Docker Hub, ECR)
-

10. Expand Cloud Infrastructure to AWS

Current State:

Cloudinary and MongoDB Atlas are already cloud-based. The backend is not yet deployed to a cloud provider.

Suggestion:

Deploy the backend on AWS using:

- **EC2** or **ECS Fargate** for compute
- **RDS** or keep **MongoDB Atlas** (managed)
- **CloudFront** + **S3** for static assets (if any)
- **ElastiCache** for Redis
- **SQS** for queues

Benefits:

- Global scalability
- High availability with multiple availability zones
- Integration with AWS AI services (Rekognition, SageMaker)

Implementation Path:

- Create an AWS account and set up IAM roles
 - Use Terraform or AWS CDK for infrastructure as code
 - Deploy using GitHub Actions
-

11. Improve Documentation Standards

Current State:

Technical documentation, API docs, and this report exist. However, they are in separate formats.

Suggestion:

Maintain documentation in a single, version-controlled system like **GitBook**, **ReadTheDocs**, or a docs/ folder in the repository with Markdown.

Include:

- Architecture diagrams (updated with each major change)
- API reference with request/response examples (auto-generated from Postman or Swagger)
- Deployment guide (step-by-step for Render/AWS)
- Troubleshooting common issues

Benefits:

- New developers can onboard quickly
 - Reduces repetitive questions to the team
 - Professional presentation to stakeholders
-

12. Strengthen Testing Practices

Current State:

Manual testing with Postman. No automated unit, integration, or load tests.

Suggestion:

- **Unit tests:** Jest for controllers and utility functions
- **Integration tests:** Supertest for API endpoints
- **Load tests:** k6 or Artillery to simulate high traffic
- **Security tests:** OWASP ZAP for vulnerability scanning

Benefits:

- Catches regressions before deployment
- Ensures stability during scaling
- Builds confidence in the codebase

Implementation Path:

- Write test scripts in a tests/ folder
- Run tests automatically in CI pipeline (GitHub Actions)
- Set a coverage threshold (e.g., 80%)

13. Optimize Mobile API Performance

Current State:

API responses return full objects (e.g., all product fields). No pagination on some lists.

Suggestion:

- Implement pagination (limit/offset or cursor) on GET /api/products and GET /api/wardrobe
- Use GraphQL (optional) to let mobile clients request only needed fields
- Compress responses with gzip (already enabled via compression middleware – good)
- Reduce image payload by using Cloudinary's responsive breakpoints

Benefits:

- Faster load times on mobile networks
 - Reduced data usage for users
 - Better Core Web Vitals scores
-

14. Future Business Expansion Opportunities

Suggestion:

Add social and community features to increase user stickiness:

- **Outfit builder** – mix and match from wardrobe
- **Fashion feed** – users share outfits (public or friends)
- **Styling assistant** – AI suggests daily outfits based on weather, occasion, and wardrobe
- **Virtual fashion show** – users can create and share virtual try-on videos

Benefits:

- Turns a utility app into a social platform
 - Increases daily active users
 - Creates new revenue streams (sponsored looks, premium filters)
-

Conclusion

Raritone already has a strong technical foundation. By implementing these suggestions – especially AI try-on, avatar generation, recommendations, Redis caching, queue processing, and robust deployment on AWS – the platform can evolve into a market-leading virtual fashion ecosystem. I am excited to see how these ideas shape the future of Raritone.